

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 9
REFACTORING**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	1
BAB I. HASIL UAS	2
I.1 Soal 1	2
I.1.A. Source Code	2
I.1.B. Analisa	3
I.2 Soal 2	4
I.2.A. Source code	4
I.2.B. Analisa	7
I.3 Soal 3	7
I.3.A. Source code	7
I.3.B. Hasil	10
I.3.C. Analisa	10
I.4 Soal 4	11
I.4.A. Source code	11
I.4.B. Analisa	12

BAB I. HASIL UAS

I.1 Soal 1

I.1.A. Source Code

```
package sebelumSolid;

public class BidangDatar {
    private int panjang;
    private int lebar;
    private int sisi;
    private double jari_jari;

    public int getPanjang() {
        return panjang;
    }

    public void setPanjang(int panjang) {
        this.panjang = panjang;
    }

    public int getLebar() {
        return lebar;
    }

    public void setLebar(int lebar) {
        this.lebar = lebar;
    }

    public int getSisi() {
        return sisi;
    }

    public void setSisi(int sisi) {
        this.sisi = sisi;
    }

    public double getJari_jari() {
        return jari_jari;
    }
}
```

```

    public void setJari_jari(int jari_jari) {
        this.jari_jari = jari_jari;
    }

    public int kelilingPersegi(){
        return 4 * this.sisi;
    }

    public int luasPersegi(){
        return this.sisi * this.sisi;
    }

    public int kelilingPersegiPanjang(){
        return (2*this.panjang) + (2*this.lebar);
    }

    public int luasPersegiPanjang(){
        return this.lebar * this.panjang;
    }

    public double kelilingLingkaran(){
        return 2*3.14*this.jari_jari;
    }

    public double luasLingkaran(){
        return 3.14*this.jari_jari*this.jari_jari;
    }
}

```

I.1.B. Analisa

Kode di atas melanggar Single Responsibility Principle (SRP) karena kelas BidangDatar mencoba melakukan terlalu banyak hal sekaligus, yaitu menangani data dan logika untuk persegi, persegi panjang, dan lingkaran dalam satu tempat. Hal ini menyebabkan kohesi yang rendah, di mana sebuah objek menjadi tidak efisien karena membawa atribut yang tidak diperlukannya; sebagai contoh, jika menginstansiasi objek untuk "Persegi", objek tersebut tetap memuat atribut panjang, lebar, dan jari_jari yang tidak relevan dan membuang memori.

Selain itu, struktur kode ini melanggar Open/Closed Principle (OCP), yang menyatakan bahwa kode harus terbuka untuk ekstensi namun tertutup untuk modifikasi. Dalam kode saat ini, jika ingin menambahkan bentuk baru (misalnya "Segitiga"), terpaksa harus mengubah (memodifikasi) kelas BidangDatar yang sudah ada, yang berisiko memunculkan bug pada fitur lain yang sudah berjalan. Desain ini membuat aplikasi sulit dirawat (maintain) dan dikembangkan (scale) untuk jangka panjang.

I.2 Soal 2

I.2.A. Source code

BidangDatar

```
package sesudahSolid;

public interface BidangDatar {
    public double getLuas();
    public double getKeliling();
}
```

PersegiPanjang

```
package sesudahSolid;

public class PersegiPanjang implements BidangDatar {
    private int panjang;
    private int lebar;

    public PersegiPanjang(int panjang, int lebar) {
        this.panjang = panjang;
        this.lebar = lebar;
    }

    public int getLebar() {
        return lebar;
    }

    public void setLebar(int lebar) {
        this.lebar = lebar;
    }
}
```

```

    public int getPanjang() {
        return panjang;
    }

    public void setPanjang(int panjang) {
        this.panjang = panjang;
    }

    @Override
    public double getLuas() {
        return panjang * lebar;
    }

    @Override
    public double getKeliling() {
        return (2 * panjang) * (2 * lebar);
    }
}

```

Persegi

```

package sesudahSolid;

public class Persegi implements BidangDatar {
    private int sisi;

    public Persegi(int sisi) {
        this.sisi = sisi;
    }

    public int getSisi() {
        return sisi;
    }

    public void setSisi(int sisi) {
        this.sisi = sisi;
    }

    @Override

```

```
    public double getLuas() {  
        return sisi * sisi;  
    }  
  
    @Override  
    public double getKeliling() {  
        return sisi * 4;  
    }  
}
```

Lingkaran

```
package sesudahSolid;  
  
public class Lingkaran implements BidangDatar{  
    private double jari_jari;  
  
    public Lingkaran(double jari_jari) {  
        this.jari_jari = jari_jari;  
    }  
  
    public double getJari_jari() {  
        return jari_jari;  
    }  
  
    public void setJari_jari(double jari_jari) {  
        this.jari_jari = jari_jari;  
    }  
  
    @Override  
    public double getLuas(){  
        return Math.PI * jari_jari * 2;  
    }  
  
    @Override  
    public double getKeliling(){  
        return Math.PI * jari_jari * jari_jari;  
    }  
}
```

I.2.B. Analisa

Secara struktur arsitektur, kode ini sudah sangat baik dan berhasil menerapkan prinsip SOLID, khususnya Single Responsibility Principle (SRP) dan Open/Closed Principle (OCP). Dengan memisahkan Persegi, PersegiPanjang, dan Lingkaran ke dalam kelas yang berbeda dan menggunakan interface BidangDatar sebagai kontrak, telah menghilangkan ketergantungan yang tidak perlu dan membuat kode menjadi modular. Jika di masa depan ingin menambahkan bangun datar baru, cukup membuat kelas baru tanpa perlu mengganggu kode yang sudah berjalan, menjadikan kode ini jauh lebih mudah dirawat (maintainable) dibandingkan versi sebelumnya.

I.3 Soal 3

I.3.A. Source code

```
import sesudahSolid.Lingkaran;
import sesudahSolid.Persegi;
import sesudahSolid.PersegiPanjang;
import sesudahSolid.Segitiga;

import java.util.Scanner;

public class BidangDatarApp {
    public static Scanner scanner = new Scanner(System.in);
    public static void main(String[] args) {

        System.out.println("Pilih bidang datar");
        System.out.println("1. Lingkaran");
        System.out.println("2. Persegi");
        System.out.println("3. Persegi Panjang");
        System.out.println("4. Segitiga");
        System.out.print("Opsi : ");
        int option = scanner.nextInt();
        if (option == 1) {
            System.out.println("Masukan jari - jari lingkaran :");
            double jari_jari = scanner.nextDouble();
            Lingkaran lingkaran = new Lingkaran(jari_jari);
```



```

        option = opsiLuasKeliling();

        if (option == 1){
            System.out.println("Kelilling    Lingkaran    :    "    +
lingkaran.getKeliling());
        }else if (option == 2){
            System.out.println("luas    lingkaran    :    "    +
lingkaran.getLuas());
        }else {
            System.out.println("Opsil salah");
        }

    }else if (option == 2) {
        System.out.println("Masukan sisi persegi :");
        int sisi = scanner.nextInt();
        Persegi persegi = new Persegi(sisi);

        option = opsiLuasKeliling();

        if (option == 1){
            System.out.println("Kelilling    Persegi    :    "    +
persegi.getKeliling());
        }else if (option == 2){
            System.out.println("Luas    Persegi    :    "    +
persegi.getLuas());
        }else {
            System.out.println("Opsil salah");
        }

    }else if (option == 3) {
        System.out.println("Masukan panjang :");
        int panjang = scanner.nextInt() ;
        System.out.println("Masukan lebar :");
        int lebar = scanner.nextInt();
        PersegiPanjang    persegiPanjang    =    new
PersegiPanjang(panjang, lebar);

        option = opsiLuasKeliling();

        if (option == 1){

```

```

        System.out.println("Kelilling Persegi Panjang : " +
persegiPanjang.getKeliling());
        }else if (option == 2){
            System.out.println("Luas Persegi Panjang : " +
persegiPanjang.getLuas());
        }else {
            System.out.println("Opsi salah");
        }

    }else if(option == 4) {
        System.out.println("Masukan alas :");
        int alas = scanner.nextInt() ;
        System.out.println("Masukan tinggi :");
        int tinggi = scanner.nextInt();
        System.out.println("Masukan sisi miring :");
        int sisi = scanner.nextInt();
        Segitiga segitiga = new Segitiga(alas, tinggi, sisi);

        option = opsiLuasKeliling();

        if (option == 1){
            System.out.println("Kelilling Persegi Panjang : " +
segitiga.getKeliling());
        }else if (option == 2){
            System.out.println("Luas Persegi Panjang : " +
segitiga.getLuas());
        }else {
            System.out.println("Opsi salah");
        }

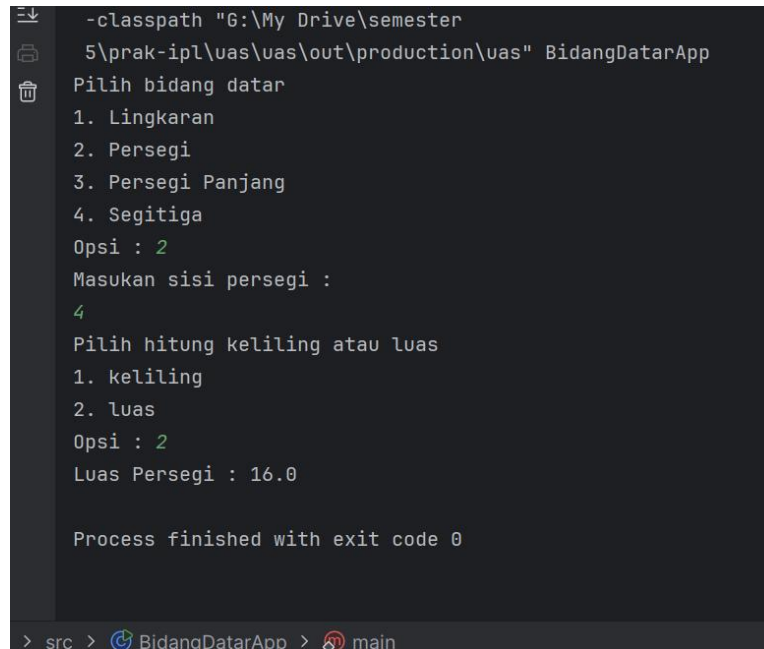
    }else {
        System.out.println("Opsi salah");
    }
}

public static int opsiLuasKeliling(){
    System.out.println("Pilih hitung keliling atau luas");
    System.out.println("1. keliling");
    System.out.println("2. luas");
    System.out.print("Opsi : ");
    return scanner.nextInt();
}

```

```
}  
  
}
```

I.3.B. Hasil



```
-classpath "G:\My Drive\semester  
5\prak-ipl\uas\uas\out\production\uas" BidangDatarApp  
Pilih bidang datar  
1. Lingkaran  
2. Persegi  
3. Persegi Panjang  
4. Segitiga  
Opsi : 2  
Masukan sisi persegi :  
4  
Pilih hitung keliling atau luas  
1. keliling  
2. luas  
Opsi : 2  
Luas Persegi : 16.0  
  
Process finished with exit code 0  
  
> src > BidangDatarApp > main
```

gambar 1. 1 Output main driver

I.3.C. Analisa

Kode ini secara fungsional berjalan, namun secara arsitektur meniadakan manfaat fleksibilitas yang seharusnya didapat dari penerapan SOLID pada kelas-kelas bangun datar sebelumnya. Penggunaan struktur if-else if yang repetitif untuk setiap jenis bangun datar melanggar Open/Closed Principle (OCP) di level aplikasi; artinya, setiap kali ada penambahan bangun datar baru, terpaksa harus memodifikasi kode di main method ini. Selain itu, kode ini belum memanfaatkan Polimorfisme secara maksimal, karena logika untuk memilih dan mencetak "Luas atau Keliling" diduplikasi berulang kali di dalam setiap blok percabangan, padahal logika tersebut identik untuk semua objek yang mengimplementasikan interface BidangDatar.

Masalah lain yang muncul adalah pelanggaran prinsip DRY (Don't Repeat Yourself) dan ketidaktepatan akibat copy-paste. menulis ulang logika pengecekan opsiLuasKeliling sebanyak empat kali, yang membuat kode menjadi panjang dan sulit dirawat. Akibat dari duplikasi manual ini terlihat jelas pada blok Segitiga, di mana output teksnya masih tertulis "Keliling Persegi Panjang" dan "Luas Persegi Panjang", yang merupakan kesalahan fatal dalam User Experience. Seharusnya, objek disimpan

dalam variabel bertipe interface BidangDatar, sehingga logika input dan output bisa dipisahkan dan diringkas.

I.4 Soal 4

I.4.A. Source code

```
package sesudahSolid;

public class Segitiga implements BidangDatar{
    private int a;
    private int b;
    private int c;

    public Segitiga(int a, int b, int c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    public int getA() {
        return a;
    }

    public void setA(int a) {
        this.a = a;
    }

    public int getB() {
        return b;
    }

    public void setB(int b) {
        this.b = b;
    }

    public int getC() {
        return c;
    }

    public void setC(int c) {
        this.c = c;
    }
}
```

```
    }

    @Override
    public double getKeliling(){
        return a + b + c;
    }

    @Override
    public double getLuas(){
        return 0.5 * a * b;
    }
}
```

I.4.B. Analisa

Secara struktural, kode ini telah memenuhi standar arsitektur yang baik dengan menerapkan Single Responsibility Principle (SRP). Kelas ini fokus sepenuhnya pada logika segitiga dan mengimplementasikan interface BidangDatar, yang memastikannya kompatibel dengan sistem polimorfisme yang Anda bangun (memenuhi Liskov Substitution Principle). Enkapsulasi data juga sudah diterapkan dengan benar melalui penggunaan modifier private pada atribut dan penyediaan metode getter/setter, menjaga keamanan akses data dari luar kelas.

